

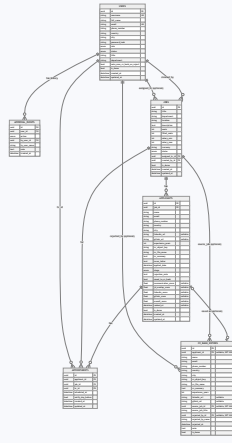
Employee Onboarding Platform

Database schema, entity relationships, authentication design, and object storage layout for the FastAPI backend powering the admin/HR onboarding and recruitment platform.

Database	Object storage	API framework	Auth
PostgreSQL 18 (onboardingplatformdb)	MinIO (S3-compatible)	FastAPI + SQLAlchemy 2.0 (async)	JWT (access + refresh), bcrypt password hashing

1. Entity-relationship diagram

Six tables cover accounts & approvals, job postings, applicants and their AI-ranking scores, interview appointments, and the CV bank. All primary keys are UUIDs generated application-side.



Design note — AI scores live on the applicant row. The four score fields plus `overall_score` are a strict 1:1 relationship with the applicant and are always read together with it, so they're embedded directly on `applicants` rather than a separate joined table. They stay NULL until "Run AI ranking" executes for the parent job.

Design note — CV bank entries are snapshots, not references. When a candidate is rejected and saved to the CV bank, their info is *copied* into `cv_bank_entries` rather than only linked. This keeps the talent-pool entry meaningful even if the original applicant or job is later deleted — the foreign keys (`applicant_id`, `source_job_id`, `rejected_by_id`) are kept as best-effort links (`ON DELETE SET NULL`) but the descriptive columns are the source of truth for display.

2. Table reference

users

Both admin and HR accounts live in one table, distinguished by role. New accounts (self-signup) start in pending status and cannot authenticate until an admin approves them; admins can also create HR accounts directly (auto-approved).

COLUMN	TYPE	NOTES
id	uuid	PK
username	varchar(50)	unique login identifier, alternative to email
full_name	varchar(150)	
email	varchar(255)	unique
phone_number	varchar(30)	nullable
country	varchar(100)	nullable
city	varchar(100)	nullable
password_hash	varchar(255)	bcrypt hash — see §3
role	enum	admin · hr
status	enum	pending · approved · rejected · removed
title	varchar(150)	nullable job title, e.g. "HR Business Partner"
department	varchar(150)	nullable
auto_save_cv_bank_on_reject	bool	HR preference; default false
is_demo	bool	flags seed/demo accounts
created_at / updated_at	timestamptz	server-managed

approval_events

Append-only audit trail: every approve/reject/remove/reinstate action on a user account, and who performed it. This is what powers the "HR approval history" view.

COLUMN	TYPE	NOTES
id	uuid	PK
user_id	uuid	FK → users.id ON DELETE CASCADE
action	enum	approved · rejected · removed · reinstated
by_user_id	uuid	FK → users.id nullable — null for the system-seeded initial admin
by_user_name	varchar(150)	snapshot of the actor's name, survives actor deletion
note	text	nullable
created_at	timestamptz	

jobs

A job posting. filled_seats increments as candidates are accepted and the job auto-closes once full. assigned_hr_id is the single HR who owns recruitment for the role.

COLUMN	TYPE	NOTES
id	uuid	PK
title / department / location	varchar	
description	text	
seats / filled_seats	int	capacity vs. hired count
salary_min / salary_max	int	
currency	varchar(10)	default "PKR"
status	enum	open · closed
assigned_hr_id	uuid	FK → users.id nullable ON DELETE SET NULL
created_by_id	uuid	FK → users.id ON DELETE RESTRICT (an admin)
is_demo	bool	
created_at / updated_at	timestampz	

applicants

A candidate who applied to a job. The CV file itself lives in MinIO — this row only stores the object key and display filename (see §4). stage drives the recruitment pipeline shown on the HR dashboard.

COLUMN	TYPE	NOTES
id	uuid	PK
job_id	uuid	FK → jobs.id ON DELETE CASCADE
name / email / phone_number	varchar	
country / city	varchar(100)	nullable
linkedin_url	varchar(500)	nullable — not every candidate has one
github_url	varchar(500)	nullable
experience_years	int	
cv_object_key	varchar(500)	nullable — MinIO object key
cv_file_name	varchar(255)	nullable — original filename for display
cv_summary	text	nullable
cover_letter	text	nullable
applied_date	timestamptz	
stage	enum	applied → assessment_passed → interview_scheduled → accepted/rejected
rejection_note	text	nullable
saved_to_cv_bank	bool	
communication_score, jd_overlap_score, linkedin_score, github_score, overall_score	float	nullable — 1–5, populated by mock AI ranking
ranked_at	timestamptz	nullable
is_demo	bool	
created_at / updated_at	timestamptz	

appointments

One scheduled interview slot for an applicant. `hr_id` is the HR who scheduled it (always the job's assigned HR in practice, enforced at the API layer).

COLUMN	TYPE	NOTES
id	uuid	PK
applicant_id	uuid	FK → applicants.id ON DELETE CASCADE
job_id	uuid	FK → jobs.id ON DELETE CASCADE
hr_id	uuid	FK → users.id ON DELETE CASCADE
scheduled_at	timestampz	
notify_day_before	bool	reminder preference — no real notification is sent yet
created_at / updated_at	timestampz	

cv_bank_entries

Snapshot of a rejected candidate's info, kept for future openings. Created automatically when an HR rejects an applicant with "save to CV bank" checked (or when their auto-save setting is on).

COLUMN	TYPE	NOTES
id	uuid	PK
applicant_id	uuid	FK → applicants.id nullable ON DELETE SET NULL
name, email, phone_number, country, city	varchar	copied from the applicant at rejection time
cv_object_key / cv_file_name	varchar	nullable
cv_summary	text	nullable
experience_years	int	
linkedin_url / github_url	varchar(500)	nullable
source_job_id	uuid	FK → jobs.id nullable ON DELETE SET NULL
source_job_title	varchar(200)	snapshot, survives job deletion
rejected_by_id	uuid	FK → users.id nullable ON DELETE SET NULL
rejected_by_name	varchar(150)	snapshot
rejected_at	timestampz	
note	text	nullable
is_demo	bool	

3. Authentication

Stateless JWT auth with an access/refresh pair, bcrypt-hashed passwords, and an admin approval gate before any new account can log in.

Signup → approval → login

1. `POST /auth/signup` creates a user with `status = pending`.

Token design

1. JWT, `HS256`, signed with a server-side secret (`JWT_SECRET_KEY`).

- 2. An admin calls `POST /users/{id}/approve` (or `/reject`), which also writes an `approval_events` row.
- 3. `POST /auth/login` checks the password, then the account status — pending/rejected/removed accounts are refused with a specific reason.
- 4. On success, the server issues a signed access token (30 min) and refresh token (7 days).

- 2. Payload: `sub` (user id), `role`, `type` (`access` / `refresh`), `iat`, `exp`.
- 3. Every protected route resolves the user via `Authorization: Bearer <token>` and re-checks `status == approved` on each request.
- 4. `POST /auth/refresh` exchanges a valid refresh token for a new pair (rotation). Tokens are stateless — there's no server-side revocation list yet.

Passwords are hashed, not encrypted. `bcrypt` with a per-password random salt is a one-way hash — the plaintext password is never recoverable, even by the platform operators. This is the correct approach for credentials (as opposed to reversible encryption, which would be appropriate for data you need to read back later, like a CV file).

4. Object storage (MinIO)

CV files never touch the Postgres database — they're uploaded straight to a MinIO bucket, and the applicant/CV-bank row only keeps the object key.

Upload path

- 1. HR/admin submits a candidate with a CV file (multipart form).
- 2. Backend generates a unique key: `cvs/<uuid>-<sanitized filename>`.
- 3. File bytes are streamed to the `candidate-cvs` bucket via the MinIO SDK.
- 4. Only the object key + original filename are saved on the `applicants` row.

Download path

- 1. Client requests `GET /applicants/{id}/cv-url`.
- 2. Backend checks the caller has access to that applicant's job.
- 3. A presigned MinIO URL is generated, valid for 15 minutes.
- 4. Client downloads directly from MinIO — the file never passes through the API server.

5. API surface (v0.1)

31 endpoints across five routers. "Who" reflects the enforced role — HR endpoints are additionally scoped to jobs/applicants/appointments the caller owns.

METHOD	PATH	WHO	PURPOSE
POST	/auth/signup	public	Request an account (starts pending)
POST	/auth/login	public	Exchange credentials for a token pair
POST	/auth/refresh	public (valid refresh token)	Rotate access/refresh tokens
GET	/auth/me	any approved user	Current profile + approval history
GET	/users	admin	List accounts (filter by role/status)
GET	/users/{id}	admin	Account detail + approval history
POST	/users/hr	admin	Create an HR account directly (auto-approved)
POST	/users/{id}/approve	admin	Approve a pending signup
POST	/users/{id}/reject	admin	Reject a pending signup
POST	/users/{id}/remove	admin	Remove an HR account (unassigns their jobs)
POST	/users/{id}/reinstate	admin	Reinstate a removed account
PATCH	/users/me/settings	hr	Toggle auto-save-to-CV-bank preference

GET	/jobs	admin (all) / hr (assigned)	List jobs
POST	/jobs	admin	Create a job posting
GET	/jobs/{id}	admin / assigned hr	Job detail
PATCH	/jobs/{id}	admin / assigned hr	Edit job details
DELETE	/jobs/{id}	admin / assigned hr	Delete a job + its applicants
POST	/jobs/{id}/assign-hr	admin	Assign/unassign the owning HR
POST	/jobs/{id}/run-ai-ranking	admin / assigned hr	Run mock AI scoring for all applicants
GET	/jobs/{id}/applicants	admin / assigned hr	List applicants for a job
POST	/jobs/{id}/applicants	admin / assigned hr	Add an applicant (+ CV upload)
GET	/applicants/{id}	admin / assigned hr	Applicant detail
PATCH	/applicants/{id}	admin / assigned hr	Edit applicant info
DELETE	/applicants/{id}	admin / assigned hr	Delete an applicant (+ CV file)
GET	/applicants/{id}/cv-url	admin / assigned hr	Presigned CV download URL
POST	/applicants/{id}/mark-assessment-passed	hr	Advance stage: applied → assessment_passed
POST	/applicants/{id}/schedule-interview	hr	Create appointment; stage → interview_scheduled
POST	/applicants/{id}/accept	hr	Stage → accepted; fills a job seat
POST	/applicants/{id}/reject	hr	Stage → rejected; optionally saves to CV bank
POST	/applicants/{id}/save-to-cv-bank	hr	Save an already-rejected applicant retroactively
GET	/appointments	admin (all) / hr (own)	List interview appointments
PATCH	/appointments/{id}/reschedule	hr (owner)	Change the scheduled time
PATCH	/appointments/{id}/toggle-notify	hr (owner)	Toggle the "notify 1 day prior" flag
GET	/cv-bank	admin / hr	Browse the shared CV bank
GET	/cv-bank/{id}/cv-url	admin / hr	Presigned CV download URL
DELETE	/cv-bank/{id}	admin / hr	Remove a CV bank entry

6. What's intentionally out of scope (v0.1)

1. Not wired to the Next.js frontend yet — the frontend still runs on its own local mock/localStorage data. This backend is a standalone, independently testable API.
 2. No refresh-token revocation/blacklist — logout is client-side only (discard the tokens). A token-blacklist table is the natural next step if that's needed.
 3. No public/unauthenticated "apply to this job" endpoint yet — applicants are currently created by an authenticated admin/HR. A public careers-page flow can be added as its own (rate-limited) endpoint later.
 4. AI ranking is a deterministic mock (hash-based), matching the frontend's simulated scoring — no model is called.
 5. Real "notify me a day prior" delivery (email/push) is not implemented — the flag is stored but nothing sends a reminder yet.